JWT 게시판 구현 보고서

이 양식은 본인이 구현한 결과를 설명하기 위한 문서입니다. 대괄호 안내를 실제 내용으로 바꾸고 작성 안내 문장은 제출 전에 지우세요. 화면 캡처와 표의 행은 필요에 따라 추가할 수 있습니다. 6~10쪽 내외를 권장하며 분량 자체는 평가하지 않습니다.

01 서비스 소개와 구현 범위

학번: 202413094 이름: 장환희 제출일: 2026/10/26

서비스 이름: 게시판

기술 조합과 버전: Vue / Express / SQLite
실행 환경: Windows / Node.js v24.15.0 / npm v11.12.1

GitHub 저장소 URL
https://github.com/202413094lab/Jang_homeWork01_Web_service_architecture.git

소스 코드 기준 [브랜치 / 코드 구현 기준 커밋 ID를 기록할 수 있습니다. 최종 제출 기준 커밋 SHA는 LMS에 제출합니다.]

[서비스 소개와 선택 이유 작성]

내가 만들 서비스는 로그인과 작성자 권한 검사를 적용한 일반 게시판을 구현하였다. 내가 일반 게시판을 구현한 이유는 내가 배운 지식을 다시 복습하고 내가 웹서비스 설계에서 배운 것에서 크게 벗어나지 않으면서 그 지식들을 사용할 수 있는 최적의 방법이 가장 기초적인 게시판을 작성하는 것이라고 생각이 들었기 때문이다. 참고로 대상 사용자는 모든 사람이다.

완료 여부는 완료·부분·미완료 중 하나로 기록하세요.

기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	T01 성공 시	app.js, store.js, api,js, App.vue	비로그인 상태에서 목록 상세 조회 가능
로그인과 JWT 발급	T02, T03 성공 시	auth.js, user.js, app.js, api.js	올바른 계정에 JWT 발급
인증과 내 정보 조회	관련 인증 테스트 성공 시	auth.js, user.js, app.js	토큰 검증 후 비밀번호 제외한 사용자 정보 반환
본인 글 작성·수정·삭제	T05~T08·T12·T17 성공 시	auth.js, app.js, store.js, api.js, App.vue	본인 글만 변경 허용
만료·로그아웃·오류 표시	T05~T08·T12·T17 성공 시	auth.js, app.js, App.vue	401 응답 시 로그인 해제하고 로그아웃 시 토큰 삭제

기능	완료 여부	관련 파일	비고 또는 제한
DB 데이터 유지	T05~T08·T12·T17 성공 시	store.js	SQLite 사용하고 서버 재시작 후 글 유지

구조와 변경 내역

요청 흐름, 토큰의 이동, 변경한 파일의 역할을 연결해 설명하세요.

서비스는 Vue로 만들어진 브라우저 화면과 Express API서버와 SQLite파일 DB로 구성된다. App.vue에서 사용자가 버튼을 누르면 api.js가 서버에 요청을 보내고 서버의 app.js가 요청을 처리한다. 게시글 데이터는 store.js가 SQLite에 저장하거나 조회한다.

로그인할 때에는 이메일과 비밀번호를 서버로 보낸다. 서버는 users.js의 verifyCredentials()로 계정을 확인하고 auth.js의 issueToken()으로 JWT를 발급한다. 프론트는 응답으로 받은 토큰과 사용자 정보를 api.js의 session변수에 저장한다. 이 값은 브라우저 메모리에 보관되므로 새로고침하면 사라진다.

글 수정 요청에는 Authorization 헤더로 토큰을 보낸다. 서버는 currentUserId()로 토큰을 검증하고 사용자를 확인한다. 이후 게시글이 존재하는지 확인하고 assertOwner()로 로그인한 사용자와 글 작성자가 같은지 검사한다. 검사 통과 시 입력값을 확인하고 DB의 글을 수정하고 결과를 반환한다.

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
auth.js/issueToken	JWT 발급	검사하고 JWT를 발급하도록 작성하였다.	계정 확인 통과 시 JWT발급하기 위해
auth.js/currentUserId	요청과 관계없이 사용자 ID 1을 반환.	헤더에서 토큰을 꺼내 검증 후 실제 계정 확인 후 사용자 ID 반환	요청 보낸 사용자 정체 확인 위해
auth.js/assertOwner()	작성자 검사 없었음.	사용자 ID와 작성자 ID 다르면 403 오류를 출력.	다른 사람이 타인의 글을 삭제 불가하도록 하기 위해
api.js/login	로그인 연결이 미구현이라는 오류를 발생	로그인을 보내고 응답의 토큰과 사용자 정보를 session에 저장하도록 작성.	화면에서 로그인 기능을 사용할 수 있게 하기 위해
api.js/authorizationHeaders	빈 객체를 반환	만약 session에 토큰이 없으면 빈 객체를 반환하고 토큰이 있으면 Bearer를 앞에 추가해 Authorization을 반환.	서버에 인증 토큰을 전달하기 위해서
api/request	실패 응답을 오류로 전달, 로그인 상태는 안 지움	만약 401이면 로그인 상태를 헤제한다.	인증 실패 시 사용자 정보 지우기 위해
api/canEdit	항상 true 반환	만약 로그인한 사용자 ID와 게시글 작성자 ID가 같을 때만 true를 반환하도록 한다.	본인 글에만 수정 삭제 버튼을 표시하기 위해

각 변경의 작성 출처 - 구현 과정에서 AI의 함수 설명, 감도 안 잡히는 곳은 AI의 수도코드, 오류 수정 안내를 참고하였다. app.js의 기본 API 처리, users.js의 비밀번호 확인, store.js의 DB저장 기능 등은 제공코드 사용.

로컬 실행 기록

실제 설치·빌드·실행 명령과 결과를 적고, 자세한 재현 방법은 저장소 README에서 연결하세요.

명령을 실행한 폴더 - backend-express, frontend-vue

SQLite 파일 경로 - backend-express/data/board.sqlite

운영체제와 버전 - Windows 11, 버전 26H2, Node.js는 v24.15.0을 사용.

환경설정

JWT_SECRET= REPLACE_WITH_RANDOM_64_HEX_CHARACTERS
JWT_ISSUER = webservice-class
JWT_AUDIENCE = board-api
TOKEN_TTL = 90
DB_FILE = data/board.sqlite
PORT = 8000

백엔드 - npm ci를 실행해 패키지 81개를 설치하였다, 이후 npm start를 실행하였다. 터미널에서는 서버가 시작되었다는 메시지가 출력되었다.

프론트엔드 - npm ci를 통해 패키지 35개를 설치했고 npm run build를 실행한 결과 Vite가 모듈 12개 처리 후 467ms 만에 빌드를 완료했다. 이후 npm run preview를 실행해 프론트 엔드를 확인 할 수 있도록 서버를 실행했다.

웹서비스설계 03

핵심 코드 설명

네 부분을 각각 설명하세요. 전체 파일을 붙이지 말고 핵심 코드 5~20줄 정도를 발췌한 뒤 입력 → 처리 → 출력과 실패 조건을 자신의 말로 설명합니다. 성공·실패 시 HTTP 상태와 JSON code, 해당 분기를 만드는 코드도 연결해 적으세요. 줄 번호를 적는 경우 제출한 코드 기준인지 확인하세요.

3 1 로그인과 JWT 발급

파일 / 함수: auth.js/issueToken

코드 발췌

```
  const sub = String(user.id);
  const exp = Math.floor(Date.now() / 1000)
    + Number(process.env.TOKEN_TTL || 90);
  if (!RandomKey) {
    throw new ApiError(500, "INTERNAL_ERROR", "서버 인증 설정 오류");
  }
  const valid = RandomKey.length === 64
    && [...RandomKey].every(c =>
```

```
      "0123456789abcdef".includes(c.toLowerCase()));
  if (!valid || RandomKey.toLowerCase() === exampleKey.toLowerCase()) {
    throw new ApiError(500, "INTERNAL_ERROR", "서버 인증 설정 오류");
  }
  const byteConvert = Buffer.from(RandomKey, "hex");
  const answer = jwt.sign(
    { sub, exp, iss, aud }, byteConvert, { algorithm: "HS256" }
  );
  return answer;
```

동작과 응답 코드 설명

로그인 검사를 받으면 app.js에서 이메일과 비밀번호를 확인한다. 입력이 잘못되면 400번 에러(BAD_INPUT)를 반환하고 계정과 비밀번호가 잘못되면 401 에러(INVALID_CREDENTIALS)를 반환한다. 만약 계정 확인에 성공하면 되면 issueToken을 호출한다. 이 함수는 서버 비밀 키가 존제하는지, 64자리 16진수인지, 제공된 예시 키와 다른지 검사한다. 만약 틀리면 500번 에러(INTERNAL_ERROR)를 던지고 맞으면 문자열 사용자 ID와 만료시각(현재 시각에 TOKEN_TTL을 더해 계산하고 기본 90초), 발급자, 사용대상을 포함하고 Buffer.from으로 32 바이트로 변환하고 HS256 서명을 적용한 토큰을 생성하여 반환한다. app.js는 로그인 성공 시 HTTP 200(OK)를 응답한다.

3 2 브라우저 토큰 저장과 전송

파일 / 함수: api.js/login, authorizationHeaders, request, canEdit

코드 발췌

1. login()

```
const data = await request("/auth/login", {method: "POST", body: databody});


session.accessToken = data.accessToken;
session.user = data.user;
return session.user;
```

2. authorizationHeaders()

```
return { Authorization: "Bearer " + session.accessToken };
```

3. request

```
if(response.status === 401)
{
     clearSession();
}
```

4. canEdit

```
if(session.user.id === post.authorId)
{
    return true;
}
```

동작과 응답 코드 설명

우선 login함수를 구현했다. 우선 email과 password를 JSON문자열로 변환한 다음 request함수의 body에 넣어 post로 request해 data에 저장한다. 그 다음 session의 accessToken에 응답의 토큰을 저장하고 응답의 사용자 정보를 session의 user에 저장하여 반환한다.
그 request 가 호출하는 함수 authorizationHeaders는 저장된 토큰이 없으면 빈 객체를 반환하고 토큰이 있으면 Authorization헤더에 Bearer + 토큰을 담은 객체를 반환한다.

request 함수 안에 response가 401이라면 ClearSession함수를 호출해 저장된 토큰과 사용자 정보를 지운다.

canEdit은 수정 삭제 버튼 표시 여부를 알려주는 함수이다. 만약 로그인한 사용자가 없으면 false, 유저 id 정보와 개시글 작성자 id 정보가 같으면 true고 다르면 false를 반환하게 한다.

서버 인증과 작성자 권한

아래 설명은 실제 제출한 파일과 함수의 동작을 근거로 작성하세요.

3 3 서버 JWT 검증

파일 / 함수: auth.js/currentUserId

코드 발췌

```
const authorization = req.get("Authorization");
const bearerHeader = authorization.split(" ")
const token = bearerHeader[1];
const key = Buffer.from(RandomKey, "hex");


let payload;
try
{
    payload = jwt.verify(token, key, {algorithms: ["HS256"], issuer: iss, audience: aud});
}
catch(error)
{

}

if(!payload || typeof payload !== "object" ||
typeof payload.exp !== "number" ||
typeof payload.iss !== "string" ||
typeof payload.aud !== "string" ||
typeof payload.sub !== "string" ||
!/^[1-9]₩d*$/.test(payload.sub) ||
!Number.isSafeInteger(Number(payload.sub))
)
{
    throw new ApiError(401, "INVALID_TOKEN", "변조·서명·클레임 오류·없는 사용자");
}

const numberUserId = Number(payload.sub);
const findUserId = findPublic(numberUserId);
return numberUserId;
}
```

동작과 응답 코드 설명

우선 request의 authorization의 헤더를 읽는다. 여기서 헤더가 없거나 값이 비어있다면 401오류, 인증 토큰 필요를 출력한다. 만약 헤더가 있다면 헤더 문자열을 공백 기준으로 나눈다.

만약 공백으로 나눈 항목이 2개가 아니거나, 소문자로 바꾼 나눠진 헤더의 첫 번째가 bearer가 아니거나, 나눠진 헤더의 두번째 항목인 토큰이 없거나 비여있으면 401 bearer형식 오류를 출력한다.

그 다음에 토큰 변수에 bearerHeader 배열의 두 번째 값인 진짜 토큰을 넣는다. 그 다음에 기존 키를16진수 문자열로 저장된 비밀 키를 바이트 형태로 변환한다.

그렇게 JWT가 유효한지 검증한다. 만약 오류가 생기면 401 변조 서명 클레임 오류 없는 사용자를 오류 메세지로 출력하고 만약 정상 서명의 토큰이 만료되었다면(error이름이 TokenExpiredError라면) 401 정상 서명의 토큰이 만료를 띄운다.

그 다음으로 payload가 없거나 비어있거나 자료형이 객체가 아니라면 혹은 각각의 JWT에 담기는 정보 항목이 각각의 자료형이 아니라면, 사용자 ID가 0이 아닌 양수로 시작하는 양의 정수 형태가 아니라면, 사용자 아이디가 안전한 정수범위가 아니라면 401 에러를 출력한다.

그 다음으론 숫자 사용자 ID를 변수에 저장하고 사용자 ID로 계정을 찾는 함수를 써서 없으면 401변조 서명 클레임 오류 없는 사용자를 출력하고 있으면 사용자 ID를 반환한다.

3 4 작성자 권한 검사

파일 / 함수: auth.js/assertOwner

코드 발췌

```
if(userId !== authorId)
{
throw new ApiError(403, "FORBIDDEN", "인증한 사용자가 다른 사람 글을 변경")
}

return true;
}
```

동작과 응답 코드 설명

JWT로 검증한 사용자 ID와 DB에 저장된 작성자 ID를 파라미터로 받고, 만약 두 ID가 다르면 403 Error, 인증한 사용자가 다른 사람 글을 변경하려고 시도함을 띄운다. 같으면 true를 반환하고 수정 삭제를 진행한다.

테스트 결과

T01부터 T17까지 수행하고, 실제 응답과 증거의 위치를 기록하세요.

실행 환경: 브라우저

로컬 배포 확인:

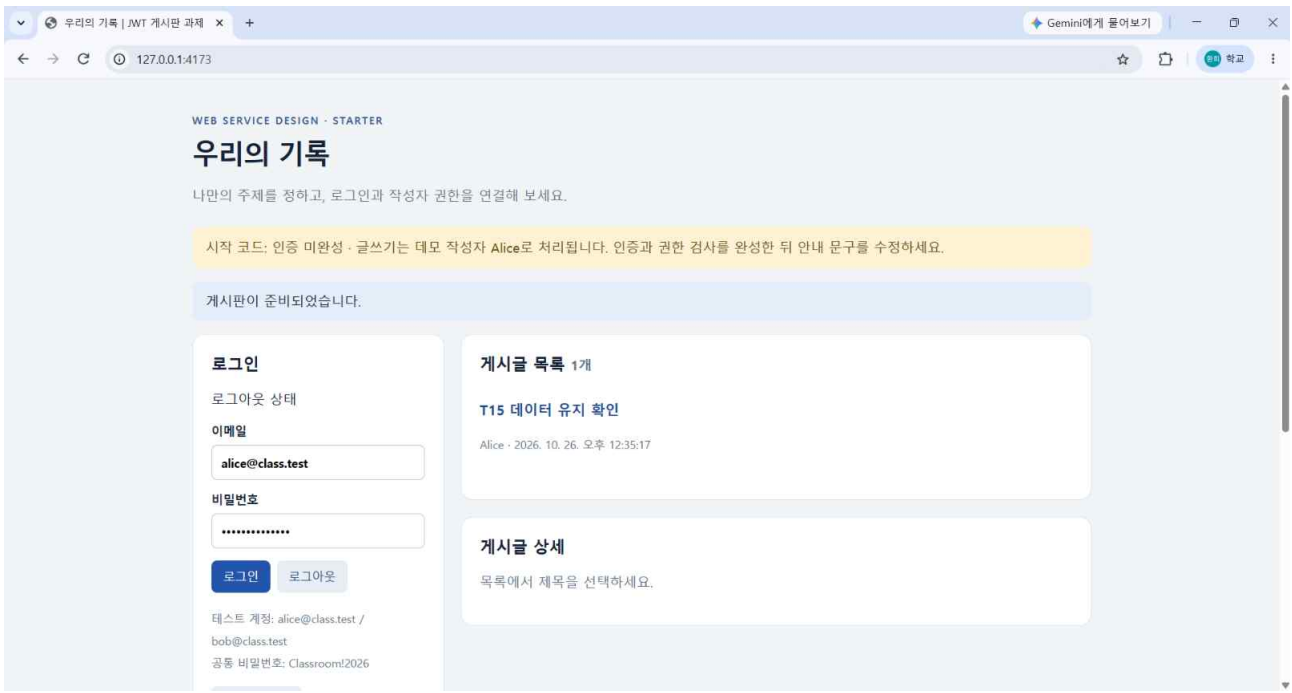
```
PS C:\Users\user\OneDrive\바탕 화면\02_시작템플릿_및_제출양식\JWT-Assignment\frontend-vue> npm ci

added 35 packages, and audited 36 packages in 2s

9 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\user\OneDrive\바탕 화면\02_시작템플릿_및_제출양식\JWT-Assignment\frontend-vue> npm run build
npm notice run jwt-board-vue@1.0.0 build
npm notice run vite build
vite v8.3.4 building client environment for production...
✓ 12 modules transformed.
computing gzip size...
dist/index.html                  0.42 kB │ gzip:  0.31 kB
dist/assets/index-DaISom2Y.css   1.65 kB │ gzip:  0.79 kB
dist/assets/index-C4j1PIMl.js   69.29 kB │ gzip: 27.47 kB

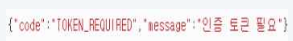
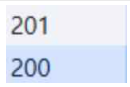
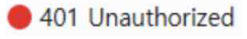
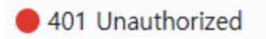
✓ built in 383ms
PS C:\Users\user\OneDrive\바탕 화면\02_시작템플릿_및_제출양식\JWT-Assignment\frontend-vue> npm run preview
npm notice run jwt-board-vue@1.0.0 preview
npm notice run vite preview --host 127.0.0.1
  ➜ Local:   http://127.0.0.1:4173/
  ➜ press h + enter to show help
```

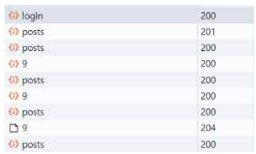


테스트 일시 및 코드 버전: 2026/10/26, Git 커밋 전 로컬 작업본

토큰 TTL: 90 / 90

05_테스트시나리오.md의 T01~T17 결과를 모두 기록하세요. PASS, FAIL, 미실행을 구분합니다. 예상 결과를 실제 결과로 복사하지 말고 직접 관찰한 HTTP 상태와 JSON code, 화면을 적으세요. 실제 결과 칸에는 예를 들어 403 / FORBIDDEN처럼 기록합니다. 정상 응답은 200 / 해당 없음처럼 적습니다.

ID	수행 절차와 사용자	예상 결과	실제 HTTP와 code	판정	증거
T01	비로그인 상태에서 게시글 목록 조회	200, 목록 정상 표시	304 해당 없음, 게시글 0개 표시.	일부 확인	
T02	올바른 이메일과 비밀번호로 로그인	200, JWT와 사용자의 정보 반환과 로그인상태 표시	200 해당 없음	PASS	
T03	잘못된 이메일 비밀번호 입력과 존제하지 않는 이메일 로그인	401 INVALID_CREDENTIALS 토큰 미발급	401 unauthorizated	PASS	
T04	로그아웃 상태에서 내 정보 조회	401 TOKEN_REQUIRED	401 Unauthorizated	PASS	
T05	Alice의 정상토큰으로 글 작성, 내 정보 조회	등록: 201, 내정보 200	등록 201, 내 정보 200	PASS	
T06	내 글 수정 삭제	수정 200, 삭제 204	수정 200, 삭제 204	PASS	
T07	bob의 유효한 JWT로 Alice 글 수정 요청 후 해당 글 재조회	403 Forbidden	403 Forbidden	PASS	
T08	Bob이 정상 토큰으로 Alice 글 삭제 요청 후 재조회	403 Forbidden	403 Forbidden 인증한 사용자가 다른 살마 글을 변경	PASS	
T09	Alice 토큰의 서명 첫 글자를 변경한 뒤 CRUD 연산	401 INVALID_TOKEN	CRUD 전부 401 Unauthorized	PASS	
T10	TTL 90초로 발급한 Alice 토큰이 만료된 뒤 CRUD 연산	모두 401 TOKEN_EXPIRED	401 Unauthorizated TOKEN_EXPIRED	PASS	
T11	Alice 로그인 후 로그아웃하고 내 정보 조회	401 TOKEN_REQUIRED	401 TOKEN_REQUIRED	PASS	

ID	수행 절차와 사용자	예상 결과	실제 HTTP와 code	판정	증거
T12	Bob의 정상 토큰으로 등록요청에 authorId:1, userId:1을 각각 추가	400 BAD_INPUT	400 BAD_INPUT, 허용된 JSON 필드만 보내세요.	PASS	400 Bad Request
T13	Alice의 정상 토큰으로 본인 글에 빈 제목, 공백 본문, 제목 101자, 내용 5001자를 각각 전송	400 BAD_INPUT	400 BAD_INPUT 빈제목title/공백본문content/제목길이초과title/내용길이초과content입력을확인하세요.	PASS	
T14	abc를 ID로 조회. Alice의 정상토큰으로 없는 글 수정/삭제.	404 NOT_FOUND 400 BAD_INPUT	404 NOT_FOUND 게시글이 없습니다/게시글이 없습니다. 400 BAD_INPUT ID는 양의 정수여야 합니다.	PASS	
T15	Alice로 글 작성후 백엔드 종료, 같은 폴더에서 재실행	기존 글과 작성자 내용 유지	기존 글과 작성자 내용 일치 / 200 OK	PASS	200 OK
T16	Alice 로그인 후 페이지 새로 고침	메모리 로그인 상태 해제, 공개 게시글 유지	로그아웃 상태인 게시판이 준비되었습니다가 뜨고 기존 글 조회 가능하다.	PASS	304 Not Modified
T17	토큰없이 기존 글 UD요청/Bob이 자신의 글 CUD	401 TOKEN_REQUIRED가 되고 글 유지/ C 201, U 200, D 204	401 TOKEN_REQUIRED 인증 토큰 필요 / 201 POST, 200 PUT DELETE 204	PASS	401 Unauthorized 

HTTP 응답 확인 기록

예상 코드와 실제 코드를 대조하고, 201·204·401의 헤더와 본문도 확인하세요.

숫자는 Network나 API 클라이언트에서 직접 관찰한 값을 기록합니다. 본문에 있는 숫자만으로 판단하지 않습니다. 한 항목이 여러 요청을 포함하면 행을 추가하세요.

확인 항목	기대 응답	실제 HTTP와 code	증거 위치
정상 조회·로그인·수정	**200**	200 / 해당 없음	T02
새 글 등록	**201 및 Location**	201 / Created	T05
본인 글 삭제	**204 및 빈 본문**	204 / No Content	T06
입력 오류·추가 작성자 필드	**400 BAD_INPUT**	400 / BAD_INPUT	T12
잘못된 계정	**401 INVALID_CREDENTIALS**	401 / unauthorizated	T03
토큰 없음·헤더 형식 오류	**401 TOKEN_REQUIRED**	401 / unauthorizated	T04
변조·클레임 오류	**401 INVALID_TOKEN**	401 / unauthorizated	T09
정상 서명의 만료 토큰	**401 TOKEN_EXPIRED**	401 / unauthorizated	T10
다른 작성자의 변경	**403 FORBIDDEN**	403 / Forbidden	T07, T08
없는 글	**404 NOT_FOUND**	404 / NOT_FOUND	T14

[보호 API 401의 WWW-Authenticate: Bearer 확인 결과와 증거]

[401과 403에서 화면이 각각 어떻게 달라지는지 설명]

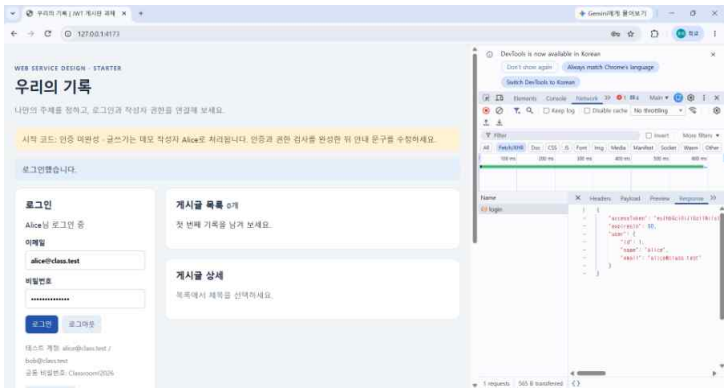
[실패한 수정·삭제 뒤 공개 조회로 데이터 불변을 확인한 결과]

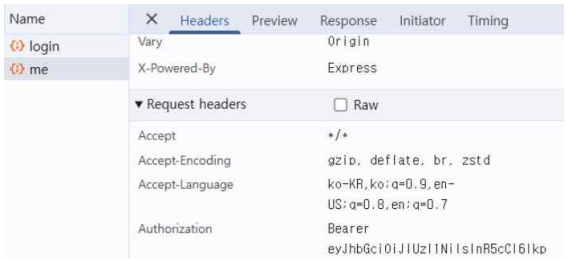
[토큰 없이 없는 글 수정 401 / 정상 토큰으로 없는 글 수정 404의 실제 결과]

테스트 증거

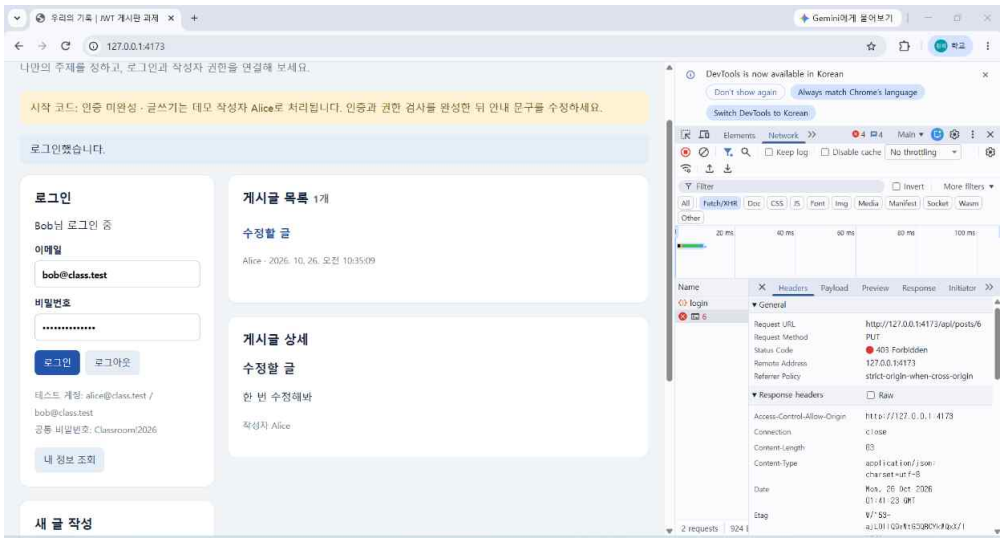
캡처에는 테스트 ID와 실제 응답을 표시하세요. 토큰 원문과 서버 비밀 키는 가립니다.

로그인과 토큰 전송

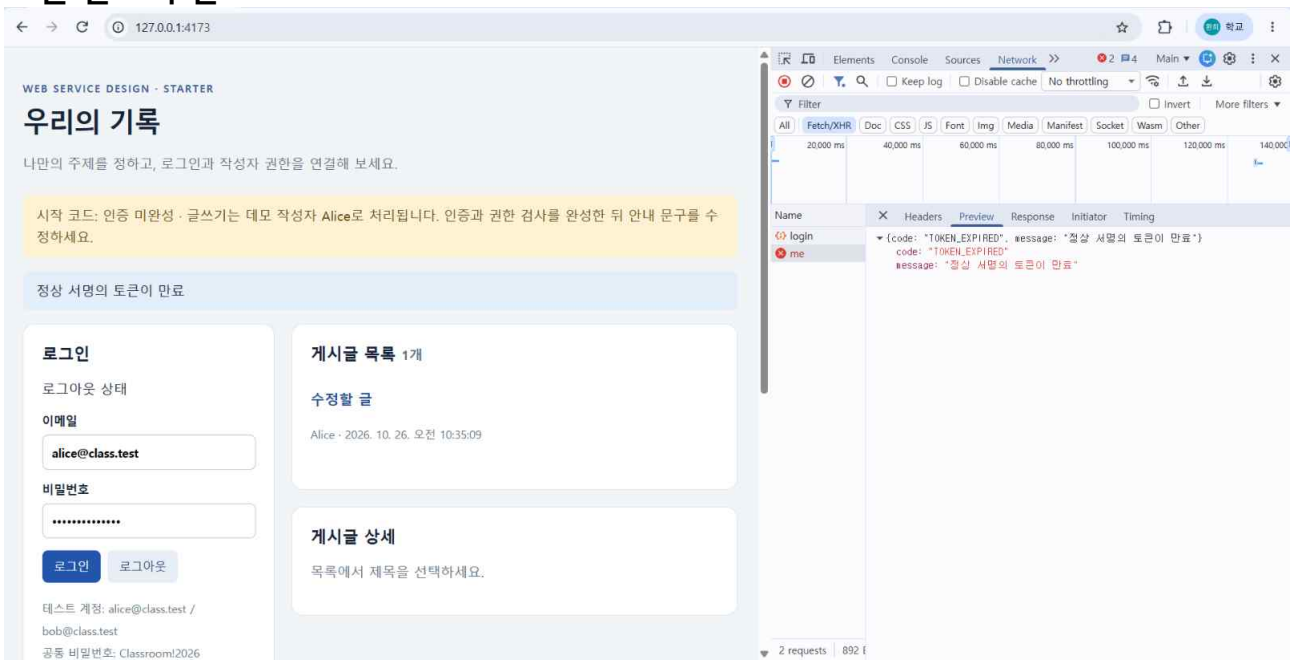


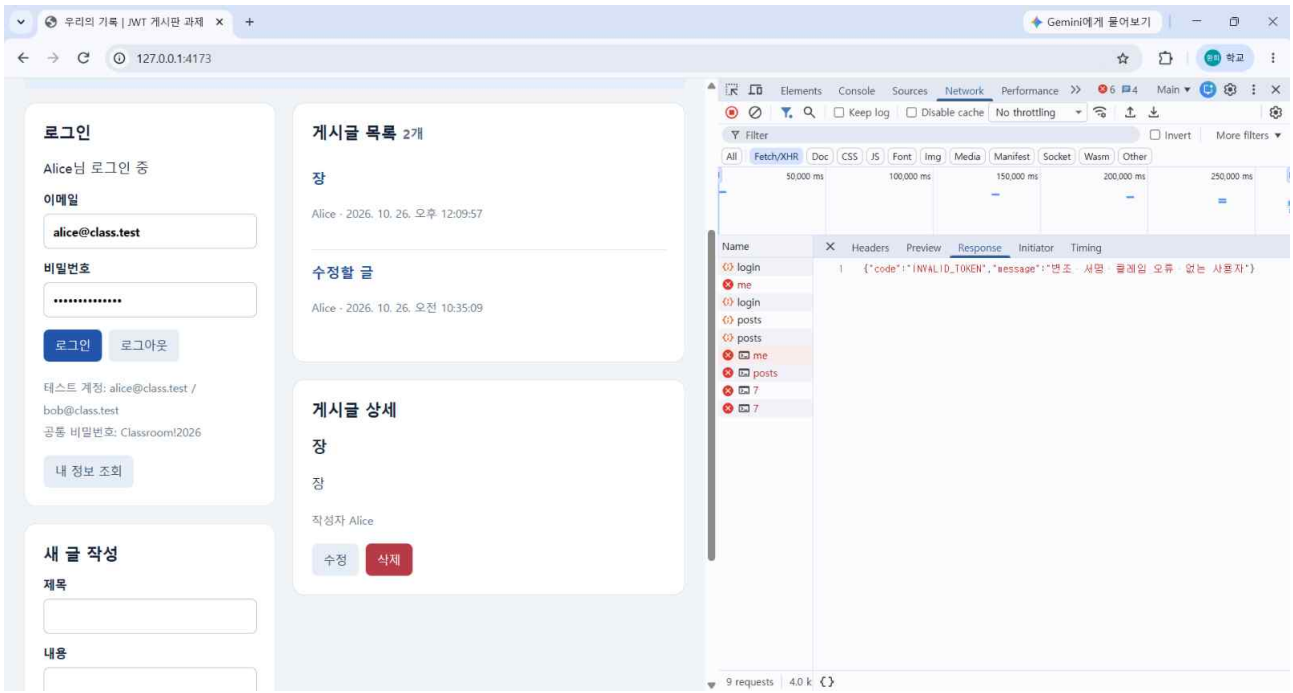


다른 작성자의 수정과 삭제



토큰 변조와 만료

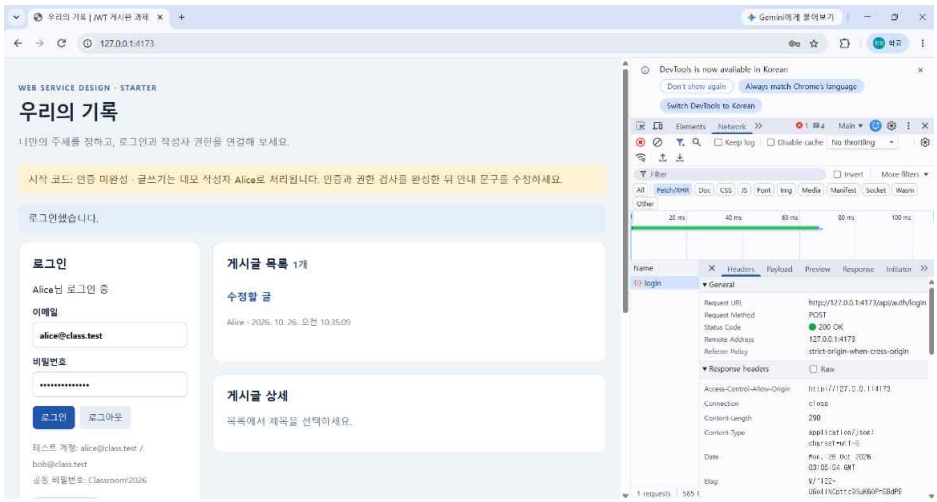

127.0.0.1:4173
WEB SERVICE DESIGN · STARTER
우리의 기록
나만의 주제를 정하고, 로그인과 작성자 권한을 연결해 보세요.
시작 코드: 인증 미완성 · 글쓰기는 데모 작성자 Alice로 처리됩니다. 인증과 권한 검사를 완성한 뒤 안내 문구를 수정하세요.
정상 서명의 토큰이 만료
로그인
로그아웃 상태
이메일
alice@class.test
비밀번호
로그인
로그아웃
테스트 계정: alice@class.test / bob@class.test
공통 비밀번호: Classroom!2026
게시글 목록 1개
수정할 글
Alice · 2026. 10. 26. 오전 10:35:09
게시글 상세
목록에서 제목을 선택하세요.
Elements
Console
Sources
Network
Preview
Response
Initiator
Timing
login
me
{code: "TOKEN_EXPIRED", message: "정상 서명의 토큰이 만료"}
code: "TOKEN_EXPIRED"
message: "정상 서명의 토큰이 만료"
2 requests


우리의 기록 | JWT 게시판 과제
Gemini에게 물어보기
127.0.0.1:4173
로그인
Alice님 로그인 중
이메일
alice@class.test
비밀번호
로그인
로그아웃
테스트 계정: alice@class.test / bob@class.test
공통 비밀번호: Classroom!2026
내 정보 조회
새 글 작성
제목
내용
게시글 목록 2개
장
Alice · 2026. 10. 26. 오후 12:09:57
수정할 글
Alice · 2026. 10. 26. 오전 10:35:09
게시글 상세
장
장
작성자 Alice
수정
삭제
Elements
Console
Sources
Network
Performance
Preview
Response
Initiator
Timing
{"code":"INVALID_TOKEN","message":"변조 서명 클레임 오류 없는 사용자"}
9 requests

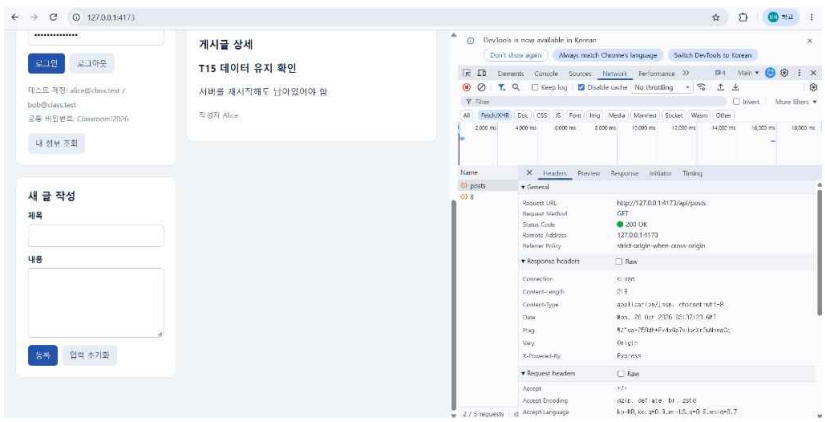
자동 검사와 로컬 배포 증거

```
PS C:\Users\user\OneDrive\바탕 화면\02_시작템플릿_및_제출양식\JWT-Assignment\frontend-vue> npm run build
npm notice run jwt-board-vue@1.0.0 build
npm notice run vite build
vite v8.3.4 building client environment for production...
✓ 12 modules transformed.
computing gzip size...
dist/index.html                   0.42 kB │ gzip:  0.31 kB
dist/assets/index-DaISom2Y.css    1.65 kB │ gzip:  0.79 kB
dist/assets/index-C4j1PIMl.js    69.29 kB │ gzip: 27.47 kB

✓ built in 383ms
```







AI 활용과 참고 자료

사용 범위, 실제 반영 위치와 직접 확인한 내용을 구분해 기록하세요.

AI 사용 여부: 사용

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식
ChatGPT, astra	학습	auth.js, api.js	참고
ChatGPT, astra	생성	auth.js, api.js	참고
ChatGPT, astra	오류 해결	.env	그대로 반영
ChatGPT, astra	테스트	보고서/테스트 결과	그대로 반영
ChatGPT, astra	문장 정리	보고서	그대로 반영
ChatGPT, astra	README 작성	README	그대로 반영

대표 사례

도구 / 날짜: GPT6 - astra

요청한 내용: 내가 떠올리지 못한 코드의 구조를 수도코드로 대신 짜 줌.

AI가 제안한 내용: 수도코드를 따라 구현하기.

실제 반영 위치: auth.js, api.js

내가 확인하고 수정한 내용: 수정하지 않았다. 애초에 수도코드이니 대충 얼개는 맞았을 뿐더러 직접 코드를 구현한 것은 나였다.

검증 방법과 결과: 실제 응답

도구 / 날짜: GPT6 - astra

요청한 내용: 테스트를 어떻게 하는지

AI가 제안한 내용: T1~T17까지 GPT가 제시해준 방향대로 하나 둘 씩 수행했다.

실제 반영 위치: 테스트 결과

내가 확인하고 수정한 내용: 수정하지 않았다. 딱히 코드를 직접 따라서 본 것도 아니고, 검증 과정이었기 때문이다.

검증 방법과 결과: 실제 응답

참고 자료

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
01_과제공지	모든 파일/함수	작업순서/필수 기능/ 검증 순서/제출

웹서비스설계 06

이해도 확인과 회고

각 질문에 본인 코드의 동작을 근거로 답하세요.

1. 요청 본문의 authorId를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

만약 Bob이 authorId를 1을 넣고 서버가 이를 믿으면 서버에서 Alice인 척할 수 있다. 그렇기에 작성자 ID를 currentUserId에서 가져온다. 요청 본문에 authorId같은 추가 필드가 들어오면 400 BAD_INPUT으로 거부한다.

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

수정 버튼은 서버에 요청을 보내는 방법 중 하나이기에 숨겨도 요청 자체를 막지는 못한다. 그럼에도 currentUserId에서 JWT를 검증하고 저장된 ID를 가져온다. 그렇게 assertOwner에서 두 아이디를 비교해서 막는다.

3. JWT decode와 verify의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

decode는 토큰의 내용을 읽는 것이고 verify는 토큰을 검증한다. 내 코드에서는 verify로 서명, 알고리즘, 만료, 발급자, 사용대상을 검증한다.

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

로그아웃은 브라우저 메모리의 토큰과 사용자 정보를 지운다, 토큰만료는 로그인 상태를 지운다, 새로고침은 메모리에 저장되었던 토큰과 사용자 정보가 사라진다. 복사한 토큰은 변하지는 않지만 새로운 토큰이 발급되면 쓰지 못한다.

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

올바른 계정으로 로그인 했지만 서버 오류가 발생했다고 했다. 도저히 답이 나오지 않아 AI의 도움으로 .env가 ㅇ벗어 JWT_SECRET을 읽지 못한다는 것을 확인했다. 그렇게 .env를 만들고 비밀키를 설정했다. 그렇게 백엔드를 제시작한 뒤 오류를 해결했다.

현재 남은 문제와 다음에 개선할 점: 이번 과제를 수행하면서 AI에게 너무 의존한 것 같다는 생각이 들었다. 다음에는 AI의 도움 없이도 할 수 있도록 실력을 키워야겠다는 생각이 들었다.

제출 확인: [ ] 실제 코드와 설명 일치 [ ] 테스트 직접 수행 [ ] AI 활용 및 출처 기록 [ ] 비밀 키와 토큰 제거

GitHub 확인: [ ] 루트 README 작성 [ ] 보고서 PDF와 증거 업로드 [ ] README 문서 링크 확인 [ ] 교수자 접근 가능 [ ] 저장소 URL과 커밋 SHA 제출